Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 2

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

```
Choose Files emails.csv
emails.csv(text/csv) - 31243156 bytes, last modified: 21/10/2026 - 100% done
Saving emails.csv to emails.csv
Dataset Shape: (5172, 3002)
  Email No.  the  to  ect  and  for  of    a  you  hou  ...  connevey  jay  \
0   Email 1    0   0    1    0    0   0    2    0    0  ...         0    0
1   Email 2    8  13   24    6    6   2  102    1   27  ...         0    0
2   Email 3    0   0    1    0    0   0    8    0    0  ...         0    0
3   Email 4    0   5   22    0    5   1   51    2   10  ...         0    0
4   Email 5    7   6   17    1    5   2   57    0    9  ...         0    0

   valued  lay  infrastructure  military  allowing  ff  dry  Prediction
0       0    0               0         0         0   0    0           0
1       0    0               0         0         0   1    0           0
2       0    0               0         0         0   0    0           0
3       0    0               0         0         0   0    0           0
4       0    0               0         0         0   1    0           0

[5 rows x 3002 columns]
```

```
X = df.iloc[:, 1:-1]
y = df.iloc[:, -1]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
print("\nClass Distribution:")
print(y.value_counts())
```

```
Features Shape: (5172, 3000)
Target Shape: (5172,)

Class Distribution:
Prediction
0    3672
1    1500
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (4137, 3000)
Testing Data: (1035, 3000)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Feature scaling completed.")
```

```
Feature scaling completed.
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("KNN Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred))
```

```
KNN Accuracy: 0.8338164251207729
KNN Confusion Matrix:
[[576 159]
 [ 13 287]]
```

```
svm = SVC(kernel="linear")
svm.fit(X_train, y_train)

svm_pred = svm.predict(X_test)

print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
print("SVM Confusion Matrix:")
print(confusion_matrix(y_test, svm_pred))
```

```
SVM Accuracy: 0.9507246376811594
SVM Confusion Matrix:
[[709  26]
 [ 25 275]]
```

```
print("Model Performance Comparison")
print("----------------------------")
print("KNN Accuracy:", round(accuracy_score(y_test, knn_pred), 4))
print("SVM Accuracy:", round(accuracy_score(y_test, svm_pred), 4))
```

```
Model Performance Comparison
----------------------------
KNN Accuracy: 0.8338
SVM Accuracy: 0.9507
```